Root-Cause Analysis — Why Reopens Stayed High and Releases Broke on First Touch

Revision: 1 **Last modified:** 2026-07-22T15:10:00Z **Status:** Phase-1 internal forensic analysis (measured-data only; the external state-of-the-art comparison is a sibling Phase-2 document) **Authority:** operator mandate 2026-07-22 (verbatim: *"do deep research on how we have for very long time so much reopened workable items!? Why the quality of delivered releases was so poor mostly? We mostly detect the same major problems on first touch when new releases are available, and this happens over and over again! … From all of this we MUST extract exact physical causes of what the gaps are, where are the weak spots and where and why and how misunderstandings still happen!?"*) · §11.4.6 · §11.4.201(6)(7)(8) · §11.4.205 · §11.4.108 · §11.4.115(F) · §11.4.146(D3) **Classification:** universal (§11.4.17) — every failure mode named here is project-agnostic; the measurements were taken in one consuming project and are anonymised per the operator's instruction. No hardware names, device serials, package names, or item ids appear in normative text; items are aliased (R1, the six operator-reopened items, …) and every number carries a parameterised measurement command in Appendix A.
[0]

. Method and anti-bluff posture

- All queries against the tracker single-source-of-truth ran `sqlite3 -readonly`. Nothing was written to any database, gate, test, or governance file.
- **Control needles (§11.4.201(7)(b)) ran before every reported absence**: the tracker path was proven non-blind (619 rows, non-zero) before any zero was reported; the guard-registry join was needled with a known-registered id; the unimplemented-gate scan was needled with a known-implemented gate

(present) and a known-deferred gate (absent) so both directions of the instrument are proven sighted. Each needle is recorded next to its measurement in Appendix A.

- **Known-unreliable instruments were avoided by name.** This repository's own trap catalogue proves `find -newermt` returns silent false-nulls here (it produced a false finding twice in one day, once inside the conductor's own check) and that the host's `grep` is a different regex engine than the habitual one, so `\|` is a literal. All file-age and existence checks below use `ls` / `stat` / `git`; all alternations use unescaped `|` under `-E`, and each load-bearing zero was needled.
- Where the data cannot answer, the finding is marked `UNKNOWN:` / `UNCONFIRMED:` (§8).

[1]

. Executive answer — the causal chain, in one page

The chronic-reopen circle is **not** produced by dishonest tests, lazy agents, or missing rules. The project's own 2026-07-17 forensics said it precisely: *"Nobody lied."* Every component was locally honest; the composition of components manufactured false beliefs. The chain, each link measured (§2–§3):

1. **A "done" status could be written by anything, with nothing checking that evidence existed** (PC-1). 48% of terminal-status items have zero audit-trail rows; a raw-write bypass of the sanctioned mutation tool was forensically proven by three independent fingerprints.
2. **Where evidence machinery existed, "never verified" and "verified good" were the same colour** (PC-2). The standing guard suite's exit code read only its FAIL count — measured live on a release candidate: 61 guards → PASS 0 / FAIL 5 / **PENDING 56** → exit 0, and the release-tag tooling consulted the suite **zero** times.
3. **Source-layer green was systematically read as done** (PC-3). 69 commits in a two-month window moved claims at "PENDING-BUILD"; fixes shipped that structurally could not execute on the deployed target; re-fixes landed in artifacts that were never flashed.

4. **The rules that forbade all of the above existed as prose, not as seams** (PC-4). The two anchors that state the fix-confirmation discipline verbatim predate the failures they failed to prevent by five-plus weeks; the corpus currently contains **85** explicit "gate-code = separate work item" deferrals and **~58% of its named gates have no implementation** in the canonical gate sites.
5. **The measuring instruments themselves produced false findings** (PC-5). 22 distinct measurement traps were catalogued from a single release cycle, hit by four independent agents; at least ten produced *reported findings* (not just wrong numbers), in both directions.
6. **Recurrences fragmented across new ids** (PC-6), so the one signal that ranks fragile work (reopen count) stayed quiet precisely on the items that kept breaking — a measured 15 new first-touch tickets collapsed to ~9 real root causes, several of them recurrences of items marked done.
7. **Guards that would have caught the defects existed and were simply never run on the deployed target** (PC-7). Two guards, when finally executed during the forensics, emitted the FAILs that had been waiting the whole time.
8. **Wrong-layer oracles certified user-visible properties** (PC-8) — a pixel defect "verified" by five text greps; harnesses twice "reconciled" to agree with the runtime.

Why the operator finds the same majors on first touch: the last audited release reached manual QA with **exactly 1 of 25** in-scope items validated at the user-visible layer. The operator was therefore not a final check — the operator was the **first** user-layer oracle the work had ever met. First-touch discovery of the same majors is the *expected output* of that pipeline shape, not bad luck; the 6/6-broken sample the operator drew from done-claiming items is the expected draw when ~95% of done-claimers carry no runnable user-layer guard.

The positive control (what held): every fix whose done-claim was preceded by an on-device RED→GREEN polarity flip on the real artifact recorded **zero** reopens; every fix confirmed only at source-green bounced. That discriminator is the project's own history selecting the formula — and it is the standard every cause below is measured against.

. The measured record (all commands + needles in Appendix A)

#	Measurement	2026-07-17 (forensic baseline)	2026-07-22 (this analysis)
M1	Tracked items / terminal-status items	576 / —	619 / 108
M2	Terminal-status items with **zero** history rows	—	**52 / 108 (48%)**
M3	Recorded reopen events vs recorded terminal events	24 / 46 = **52%**	25 / 51 = **49%**
M4	Items in status Reopened with **no** reopen event recorded	14 / 27	**13 / 26**
M5	The worst item (alias **R1**): reopen events / terminal events	4 / 0	4 / 0 (unchanged)
M6	Reopen-event attribution	—	**25/25 = "AI"; 0 = "User"** — while the six operator-driven reopens carry no events at all
M7	Done-claiming items / of them guarded (id-or-alias join)	182 / 9 (**95% unguarded**)	187 / 14 (**92.5% unguarded**)
M8	Guard registry rows / distinct lead keys / item-shaped keys	61 / 22 / —	90 / 59 / 37 (**22 keys are not item ids** — the join gap persists)

#	Measurement	2026-07-17 (forensic baseline)	2026-07-22 (this analysis)
M9	Suite semantics on PENDING	PENDING **does not** affect exit (live: 61 → 0 PASS / 5 FAIL / 56 PENDING → **exit 0**)	PENDING → **exit 3, blocks** (landed 2026-07-18)
M10	Release-tag tooling consults the guard/verdict layer	**0 references**	verdict-coverage gate wired (landed 2026-07-18)
M11	Status-write custody seam (refuse write without evidence chain)	designed	**NOT implemented** — custody sweep script absent, custody gate absent from both canonical gate sites, adoption-ratchet file absent (all needled)
M12	Rule corpus	—	10,735-line canonical rule file; **220 distinct** §11.4.N anchors (max index 224) + 10 §12.N; **85** literal "gate-code = separate work item" deferrals; **413** named CM-* gates, **241 (58%)** with no implementation in either canonical gate site
M13	Governance-mirror integrity	—	**two anchor blocks duplicated verbatim** (each appears 2×) in the canonical agent-facing mirror; undetected because propagation gates assert literal-presence ≥ 1
M14	First-touch QA outcome (last audited release)	—	25 in-scope items; **1 CLOSED-VALIDATED**; 2 tracker-terminal statuses flagged BLUFF-RISK; 19 newly filed

#	Measurement	2026-07-17 (forensic baseline)	2026-07-22 (this analysis)
M15	Recurrence compression	—	15 new first-touch tickets ≈ **9 root causes**; 4 high-confidence recurrences of "done" items; 1 pair the operator suspected same proven **distinct**
M16	Git pathologies (2-month window)	1,370 commits; 202 fix-commits; **69** "PENDING-BUILD" claim-moves; **81** gate-reconciliation citations (**0.40 per fix commit**)	—
M17	Instrument traps catalogued (one release cycle)	—	**22 traps** + 9 derived rules; 4 independent agents each produced ≥1 false finding; one **environmental root cause** (host grep is a different regex engine) unified ≥4 "independent" mistakes
M18	Oracle wiring census (behavioural)	—	68 oracles; **24 referenced by no seam; 12 with no caller anywhere**; **79 guards** omit the evidence-init call so their evidence artifact is never written
M19	Doc reachability from the mandated entry point	—	1,953 orphan docs (422 after index repair); the two "single canonical entry point" mandates **contradict each other** (the mandated resumption file was unreachable from the mandated README)

. The ranked physical causes

Each cause is stated as: the **mechanism** (what physically happens), the **evidence**, **why the existing rule failed to prevent it**, and the **class** (the universal name of the failure mode).

PC- — The status field had no custody: "done" could be written with nothing checking that evidence existed

Mechanism. The tracker's `status` column was writable by any process — including raw SQL that bypassed the sanctioned mutation tool. The sanctioned tool writes status + audit row + body atomically; a raw `UPDATE` writes only the column. Nothing at the write seam refused the un-evidenced write, and nothing at the build or release seam swept for it.

Evidence. (a) 52 of 108 terminal-status items have **zero** history rows of any kind [M2]; (b) 13 of 26 currently-Reopened items have no Reopened event [M4]; (c) item R1 carries 4 reopen events and 0 terminal events — an impossible sequence (you cannot reopen what was never closed) proving the terminal transitions bypassed the ledger [M5]; (d) the 2026-07-17 status-custody investigation **proved** a raw-SQL bypass by three independent fingerprints: zero correlated history rows, a stale cached body the sanctioned tool always regenerates, and a bare-date `last_modified` that the tool's `datetime('now')` can never produce. (e) The bias finding [M6]: all 25 *recorded* reopen events are attributed to the in-loop AI channel, while the six operator-driven reopens — the highest-severity escapes in the record — left **no** custody trail. The intake channel that catches the worst escapes is the one least likely to be recorded, so the recorded ledger systematically understates exactly the evidence the operator's question is about.

Why the rules failed. §11.4.146 and §11.4.115 stated the required chain verbatim — and were prose an agent consults, not a condition a seam checks (the §11.4.205 test fails). §11.4.34 required reopen attribution; nothing refused a status write lacking it.

Class. *Unmediated write to a single source of truth* — any SSoT writable outside its audited mutation path will, under deadline pressure, be written outside its audited mutation path.

PC- — Absence of verification was indistinguishable from verification

Mechanism. The verdict vocabulary had three real states (verified-good, verified-bad, never-verified) but the seam collapsed them to two: only FAIL affected the exit code. A guard that could not run (fix not built, device absent, asset missing) produced PENDING, and PENDING was invisible.

Evidence. Measured live on a flashed release candidate: registry 61 → PASS **0** / FAIL 5 / PENDING **56** → **exit 0** "not blocked" [M9]. The release-tag tooling referenced the guard suite **zero** times [M10] — so even the FAIL count was consulted by nobody at the moment of tagging. Downstream of the same mechanism: 79 guards omitted the evidence-initialisation call, so their evidence artifact was silently never written [M18] — evidence-absence again reading as no-signal rather than as failure. The complete causal chain (from the project's own forensics): honest guard written → fix never flashed → guard PENDING → PENDING invisible → item marked done → operator finds it broken → reopen. **No component lied.**

Why the rules failed. The anti-bluff covenant forbade PASS-bluffs and FAIL-bluffs — but "exit 0 with 56 PENDING" is neither: it is a *coverage* hole, a category the rule vocabulary did not name until 2026-07-17. A naive "PENDING always blocks" would have been a FAIL-bluff that got the gate switched off; the correct seam semantics (block on missing coverage **of the release candidate's fingerprint**, stay honest on intermediate artifacts) had to be invented.

Class.[3] *Null-as-success at a decision seam* — the most dangerous polarity, because a broken instrument and a healthy system return the identical quiet zero.

PC- — Verification layers were conflated: source-green was read as done

Mechanism. A fix crosses four layers (source → artifact → runtime-on-clean-target → user-visible). The tracker's status vocabulary and the agents' working habit both allowed a claim to move on layer-1 evidence alone.

Evidence. 69 commits in a two-month window moved claims at "PENDING-BUILD" [M16]. Concrete measured instances from the reopened set: a fix whose code **structurally could not execute** on the deployed image (a permission model prevented the writing process from ever registering — the slider it fixed was a no-op

end-to-end, undetectable at source layer); a config flag asserted green against **a file the build does not read** (artifact-layer divergence); a codec shipped `enabled="false"` in the artifact while the source claim was green; re-fixes landed in an artifact that was **never flashed**, then the same defects were re-reported by the operator on first touch. The last audited release's QA entered with 1/25 items validated past layer 1 [M14].

Why the rules failed. §11.4.108 named the four layers precisely — and every one of the above happened *after* it was landed. The layer discipline existed as a classification agents applied in prose ("SOURCE-COMPLETE / PENDING-BUILD") — the honest label was even used! — but no seam refused a *terminal status* carrying only a layer-1 label. The honest vocabulary coexisted with the dishonest state transition.

Class. *Layer conflation with honest labels* — a system can name its layers correctly in every document and still let the status machine ignore them.

PC- — The rules existed as prose, not as seams (and the remediation partially repeated the pattern)

Mechanism. A rule stated in governance text binds only if some executable checks it at a write seam. Authoring an anchor is cheap (minutes); implementing its seam is expensive (hours to days). Under that asymmetry the corpus's rule-count grows faster than its seam-count, and the gap *is* the defect surface.

Evidence. The corpus contains **85** literal "gate-code = separate work item" deferrals and **413** named gates of which **241 (58%)** have no implementation in either canonical gate site [M12] (needled in both directions — a known-implemented gate resolves, a known-deferred one does not; residual implementations in non-canonical seams are possible and marked in §8). The two anchors that state the fix-confirmation discipline verbatim predate the measured failures by five-plus weeks. **And the remediation itself partially remained prose:** of the three custody seams designed on 2026-07-17, the release seam landed within a day (suite PENDING-blocks; verdict-coverage wired into the tag tool [M9, M10]) — but as of this analysis the **status-write refusal and the full-table custody sweep do not exist, and the adoption-ratchet file was never created**

[M11]. Five days after the forensics that diagnosed "rules as prose," the highest-value two of its three prescribed seams are still prose. Meanwhile the unguarded-done ratio moved only from 95% to 92.5% [M7].

Why the rules failed. §11.4.205 itself now names the test ("if I documented this and nobody implemented the hook, would anything catch it?") — and §11.4.205 postdates most of the corpus. The propagation-gate class that *does* run everywhere enforces the presence of anchor **text**, mechanically rewarding restatement over implementation (see §5).

Class.[5] *Prescription–enforcement decoupling* — governance that measures its own propagation in words per file will optimise words per file.

PC- — The measuring instruments themselves produced false findings

Mechanism. Every load-bearing measurement crosses a path (shell quoting, regex dialect, pipeline exit status, line anchoring, encoding, tool substitution), and each layer can return a clean, confident, wrong answer without erroring. A false null (silence read as absence) and a false match (a carrier that *mentions* X matched as X) are invisible to parse checks and code review.

Evidence. 22 traps catalogued from one release cycle, hit by four independent agents [M17]. Counting only corrupted **findings** (statements briefed or reported, not private wrong numbers): a false "recorder not running — mandate unmet" report; a false "registry is empty" (it had 6 rows); two mutations falsely reported as gate-missed (they had never applied); a release-line comparison against the wrong tag producing a clean, meaningless verdict that pointed the *convenient* way; a whole-cycle "file unchanged, zero footprint" summary while the file carried +742 uncommitted lines; a briefing of 3 pre-build blockers when there were 5; a false "0 credential offenders" from a mis-replicated OR-gate; a background wrapper reporting exit-0 for a build that failed; and — inside the release validation procedure itself — a constant-false check, a CRLF truncation, a false-FAIL from a missing device selector, and **two false PASSes** in successive review rounds. **≥ 10 reported findings corrupted in one cycle, in both directions.** The deepest instance: what had been catalogued as four independent agent mistakes was proven to be **one**

environment fact (the host's `grep` is a different engine, making a common escape a literal) — a single instrument substitution silently biasing every agent's measurements until a control needle exposed it.

Why the rules failed. The anti-bluff covenant governed *test* verdicts; nothing governed *measurement* verdicts until §11.4.201(6)–(8) was extended (2026-07-17). Parse-checking (§11.4.67) runs at parse time; the FAIL-crash clause (§11.4.1) covers crashes — none of these traps crash. Only the control-needle discipline detects them, and it was adopted mid-cycle, after the damage.

Class. *The path is part of the instrument* — an unvalidated instrument is not an instrument, and its most dangerous output is a quiet zero.

PC- — Recurrences fragmented across new ids, silencing the fragility signal

Mechanism. When a defect recurred, the recurrence entered the tracker as a **new** id (sometimes *in addition to* an unaudited status flip on the old id — double-tracking). The original item's reopen counter therefore never incremented; the metric used to rank the most fragile work for the deepest scrutiny stayed quiet precisely on the items that kept breaking.

Evidence. 15 first-touch tickets ≈ 9 root causes [M15]; measured family chains where a "done" item's defect re-entered under a fresh id (an overlay defect chain spanning three ids is documented **in a source comment in the shipping code**, which names the chain explicitly — the code knew what the tracker did not). The recorded 52% reopen rate is therefore a proven **undercount** [M3 + M15]. One suspected same-defect pair was proven *distinct* on close reading — which is the counter-case that justifies why naive auto-merging would be worse (a wrong merge silently deletes a real defect); the mandated default (link-with-candidate, reopen only through the duplicate-chain head) postdates these measurements.

Why the rules failed. The dedup rule (§11.4.186) gated *export/sync/commit*, not *intake*; the reopen-attribution rule (§11.4.34) presupposed the status was already flipped; the recurrence-links-not-mints rule (§11.4.214) did not exist until 2026-07-17. The intake seam that mattered — "before minting, resolve against existing items" — had no rule and no mechanism.

Class. *Identity fragmentation starves the prioritisation signal* — a feedback loop where the metric that should focus attention is silenced by the very failures it should count.

PC- — Guards existed and were simply never run where the defect lives

Mechanism. Guard authorship was treated as coverage. A registered guard that never executes on the deployed target contributes exactly nothing except a green feeling; worse, mixed registry keys made the item→guard join fail, so even class-level coverage was invisible to the item.

Evidence. During the forensics, two standing guards were finally executed against the flashed target and immediately emitted the FAILs that had been latent the whole time (one reproduced the operator's loudest defect 3/3). 22 of the registry's 59 lead keys are still not item-shaped ids [M8], so a by-id custody join misses them. The behavioural oracle census found 68 oracles with 24 referenced by **no seam** and 12 with **no caller anywhere** [M18]. Guard *viability* was also unproven: one long-standing test set a PASS floor above the size of the roster it counts — it was structurally incapable of passing for its entire life, and nobody noticed because it was never required to produce a verdict. And author-side validation is structurally insufficient: in one measured case an author's six self-authored mutations were all caught while an independent reviewer's **first three** different mutations all sailed through, each restoring a user-visible defect.

Why the rules failed. The standing-guard mandate (§11.4.135) required guards to *exist*; existence was auditable, execution was not — until verdict files keyed to the candidate's fingerprint were mandated (2026-07-17). A guard never observed failing on the genuinely broken artifact is unvalidated instrumentation (the RED run is the fault-injection that validates the detector), and nothing demanded that observation.

Class. *Registered ≠ executed; authored ≠ validated* — the two cheapest states of a guard are the two that satisfy a presence-audit.

PC- — Wrong-layer oracles certified user-visible properties, and harnesses were bent toward the runtime

Mechanism. An oracle observing a proxy layer (source text) cannot distinguish correct from incorrect behaviour at the defect's layer (pixels, audio, routing). Separately, when a gate FAILed after a change, the cheap resolution was to rewrite the gate to agree with the runtime — twice in one fix-chain the harness, not the product, was "reconciled."

Evidence. The canonical instance: a pixel-layer defect (an overlay covering the screen) certified "fixed" by a gate composed of a file-existence check and five greps — which also guarded only one of the two independent enable-paths (a multi-factor gap; flipping the unguarded path leaves the gate green). 81 commits in two months cite gate-reconciliation — **0.40 reconciliations per fix commit** [M16]: at that rate the gate population is being continuously re-authored to match the code, the inverse of the code being verified against the gates. The token-blindness class (a gate asserting a *symbol appears somewhere* instead of the structure) recurred **five times, across three artifacts, by four different agents, in a single cycle**.

Why the rules failed. Evidence-class matching (pixel defect ⇒ pixel evidence) was implicit in the captured-evidence anchors but not *refusable* — no seam rejected a verdict whose evidence files were of the wrong class until the evidence-shape whitelist was designed (2026-07-17). Legitimate gate-reconciliation (§11.4.120) exists, which made illegitimate reconciliation cheap to rationalise.

Class. *Oracle–defect layer mismatch* plus *instrument capture* — when the test can be edited more cheaply than the product, sustained pressure edits the test.

PC- — The done-vocabulary itself was ambiguous, and operator and machine read it differently

Mechanism. A status that reads as "done, awaiting a formality" to a human ("Ready for testing") functioned in practice as design-complete-but-**unverified**. 79 items — 42% of the done-claiming population — sit in that status today [M7 basis]. The operator sampling "done-claiming" items was sampling a set dominated by never-verified work without either party being wrong about their own definition.

Evidence. The forensic audit of the six operator-reopened items found every one had been sitting in this status with **no terminal event and no on-device confirmation ever recorded**; the custody investigation had to explicitly argue that reverting them to it "would restore a false positive, not a neutral state." The done-claiming definition used in the 95%-unguarded arithmetic *includes* this status — correctly, because it is the state from which items were experienced by the operator as claims.

Why the rules failed. The status vocabulary (§11.4.15) defined lifecycle order but assigned no **evidence precondition** to each state; the type-aware closure vocabulary (§11.4.33) governed the final word, not the penultimate state where most of the population parks.

Class. *Semantic drift between claim-maker and claim-consumer* — the cheapest state that sounds like progress becomes the parking lot for unverified work.[4]

. Where and why MISUNDERSTANDINGS still happen — the human/agent layer

The operator asked specifically where misunderstanding survives a rule corpus this explicit. The evidence shows six repeatable shapes. The unifying observation: **misunderstandings are manufactured at composition seams between honest components** — between a rule and its scope, between a measurement and its consumer, between one session's fact and the next session's premise.

MU-1 — Premises inherited without re-measurement. A design document asserted "22 edit points" for a removal; the measured count at execution time is **45**, and the design figure is flagged UNVERIFIED in the live handoff. A "next free anchor number" was correct when measured and stale when used (the number had been consumed in between), producing a duplicate-numbering near-miss. A live-state table carried three stale rows (all three corrected on re-verification). The mechanism: a fact is measured once, becomes text, and text does not expire. Text that encodes a *measurement* needs a re-measurement trigger, and prose has none — only machine-derived blocks with staleness detection (§11.4.205(4)) close this.

MU-2 — A true verdict leaking beyond its scope. A structurally-correct impossibility verdict ("cannot relocate another app's protected surface *without its cooperation*") was cited for six weeks against an **adjacent, viable** goal (whole-task placement) — while the platform's own documentation prescribed a public API for the adjacent goal. Nothing bounded the verdict's reach: the research rule fired where verdicts are *minted*, and this failure happened where one was *cited as a premise*, which crosses no closure seam. (Now §11.4.112(5): scope statement + adjacent-goals enumeration + citation-fencing.)

MU-3 — A gate imposed on the wrong seam. Work was ordered "run checks E1–E4, then implement P1" — while the gate's own definition required E2 to run **on a P1-gated build**. The gate's source-of-truth seated it at *enabling*; a restatement moved it to *authorship*, creating a deadlock that read as "blocked" (§11.4.120 seam-placement extension). The mechanism: rules are restated as they travel (brief → plan → dispatch), and each restatement is an opportunity to move a load-bearing qualifier.

MU-4 — Two canonical rules, each individually reasonable, contradicting in composition. The standing-resumption-file mandate designates one file as *the* out-of-the-box entry point; the README-entry-point mandate designates another. Measured: the mandated resumption file was **unreachable** from the mandated README [M19] — the audit's own words: "the two mandates contradict each other today." A corpus this large composes rules faster than any author re-checks pairwise consistency; nothing mechanical checks inter-anchor consistency at all (§5).

MU-5 — Right behaviour, wrong mechanism — and a wrong mechanism yields a wrong remedy. The regex-dialect discovery was first reported with a false mechanism (a repo-shadowed binary that does not exist) before being corrected to the true one (a host-wide tool substitution). Under the false mechanism the remedy would have been to hunt a nonexistent file and to wrongly assume other checkouts were safe. Mis-attribution also appeared at the defect level: a reopened item was attributed to one overlay mechanism when the operator had seen a *different* overlay whose chain the source code itself documents — the misread survived because attribution was done from symptom-words, not from the mechanism inventory.

MU-6 — Counts read as state. Twice in one cycle a *count* was converted into a *finding* without reading the underlying lines (a "mixed half-written state" that was actually a coherent doc with a labelled historical citation; an inflated FAIL total that included the summary line counting itself). The catalogue's rule is exact: *a count is a lead; the lines are the finding* — and a summary line is a carrier of the thing it summarises.

Cross-cutting driver. All six shapes worsen superlinearly with corpus volume and session turnover: more anchors → more restatements in flight → more chances for MU-2/MU-3 scope drift; more handoffs → more inherited premises (MU-1); more prose mirrors → more divergence surface (MU-4, and the duplicated anchor blocks in M13). This is the honest bridge to §5.
[5]

. The rule corpus itself, honestly

The corpus is empirically-derived, mostly correct, and unusually candid — most anchors carry a real forensic incident. **And it is, at its current scale and enforcement ratio, part of the problem.** The evidence:

1. **Volume.** ~230 anchors (220 distinct §11.4.N + 10 §12.N) in a 10,735-line canonical file [M12], mirrored into 4+ agent-facing files; three of the mirrors alone total **8,092 lines** of governance context loaded before any work begins. UNKNOWN: the token-budget crowding effect on agent behaviour is not directly measurable here — but MU-1/MU-2/MU-3 are its observable signature: the more text an agent must hold, the more it operates on remembered summaries of rules, which is precisely where scope qualifiers fall off.
2. **The enforcement ratio is the real defect, and it is measured.** 85 explicit "gate-code = separate work item" deferrals; 241/413 named gates (58%) with no implementation in the canonical gate sites; the central custody seam designed five days ago still unimplemented [M11, M12]. The corpus's own strongest recent anchor (§11.4.205) states the criterion that convicts it: a rule documented without a hook, where nothing catches the absence, is "WORSE than no rule — it terminates the scrutiny that would have caught it." By that test, roughly half the corpus's named enforcement surface is currently in the worse-than-nothing state.

3. **The propagation-gate class optimises the wrong variable.** The one gate family that runs everywhere checks the presence of the **anchor literal** in each mirror file. It mechanically rewards restatement (text growth) and cannot see: duplication (two anchors appear **twice**, verbatim-divergent, in the canonical mirror [M13] — undetected because presence ≥ 1 is satisfied), drift between copies, or contradiction between anchors (MU-4). The corpus measures its own health in words present, and so it grows words.

4. **Redundancy and near-duplication are real.** Measured examples: the most-reopened-first priority exists twice (as ranking factor (d) of one anchor and as a standalone anchor); the README-entry-point anchor is a self-described generalisation of an earlier anchor that remains in force; the fix-confirmation discipline exists in at least three anchors (TDD-fix, RED- polarity, reproduce-first) whose non-overlap no seam checks; two anchors govern "single entry point" and contradict (MU-4). Each redundant statement is one more restatement surface for MU-class drift.

5. **The growth loop is incident-driven and one-directional.** The observable pattern in the corpus's own revision log: incident → new anchor (same week) → propagation gate (text) → gate- code deferred. Anchors function as **incident memorials**; memorials accumulate; seams lag. Nothing retires, merges, or consolidates anchors (one repeal exists in ~230), and no metric tracks anchors-per-seam. The honest statement of the finding: **the corpus does not fail by being wrong; it fails by out-running its own enforcement and by having no consolidation pressure.** A rule corpus needs the same custody discipline it imposes on fixes: an anchor's "done" state should be its seam landing, not its text landing — and by that standard most of the corpus is in "Ready for testing."

6. **What the corpus got right, measured.** The discipline that *was* mechanised worked: the suite's PENDING semantics and the release verdict-coverage seam landed within a day of diagnosis and now block [M9, M10]; the audited mutation tool is structurally incapable of the custody-free write (the bypass had to go *around* it, and its fingerprints made the forensics possible); the control-needle rule, once adopted, caught real traps within days, including in the conductor's own checks. Every fully-mechanised rule in this record either prevented or exposed its failure mode. The gap is not rule quality; it is the prose-to-seam conversion rate.

. Why the same major problems surface on first operator touch — the synthesis

Combining PC-1…PC-9: at release time, the *pipeline's own* user-layer verification had run for almost none of the done-claiming population (1/25 in the audited release [M14]; ~93–95% unguarded [M7]; PENDING invisible [M9]; guards unexecuted [PC-7]). The operator's first touch was therefore the **first execution of the user-layer oracle** for most claims. The "same" problems recur because (a) the underlying defect families were never closed at their layer (PC-3, PC-8), (b) their recurrence entered under new ids so nothing ranked them as fragile (PC-6), and (c) the re-fixes repeatedly landed in artifacts that never reached the operator's device (PC-3). Under those conditions, first-touch rediscovery of the same majors is not an anomaly to explain — it is the deterministic output of the measured pipeline shape. The corrective invariant is the one the project's own history selected: **no done-claim before an on-device RED→GREEN polarity flip on the shipping artifact, and absence of a verdict blocks exactly as a FAIL does** — with both halves enforced at seams (status-write, build, release), not stated in prose. Two of those three seams remain unimplemented at the time of this analysis [M11].
[7]

. The positive control — what held

Every fix whose done-claim was preceded by an on-device RED→GREEN flip that *held* recorded **zero** reopens; every fix confirmed at source-green bounced (the project's 2026-07-17 discriminator, re-verified against today's ledger: the guarded-and-flipped closures remain uncontradicted). Honest confound, carried forward: most guarded items have had less operator exposure (partially censored data), and the clean exemplars number two — the correlation is strong and mechanistically explained, not a randomized trial.

. UNKNOWN / UNCONFIRMED register

- **UNKNOWN: which process issued the raw status writes.** No shell history or process log survives; only the three side-effect fingerprints. (This is itself the argument for write-seam enforcement over post-hoc forensics.)
- **UNCONFIRMED: implementations of the 241 "unimplemented" gates outside the two canonical gate sites.** The scan covered the pre-build suite and the constitution gate directory; commit hooks and per-script seams were spot-checked only (one high-profile 2026-07-17 gate: zero hits in all script trees searched; a full-tree scan timed out and was not retried against the multi-GB source tree). What would settle it: a registry mapping every named gate to its implementing file, itself gated — which is the §5 recommendation in miniature.
- **UNCONFIRMED: the "guide naming the wrong device for months" example** cited in the tasking. This analysis did not locate the specific document in the time available; the adjacent, verified instances (stale live-state table with three wrong rows; stale design counts) carry the same class. What would settle it: the conductor's citation of the specific guide + its git history.
- **UNKNOWN: the true reopen rate.** 49–52% is a proven undercount (PC-6); the family-chain analysis suggests the real rate is materially higher, but a defensible corrected figure requires the duplicate-chain reconciliation to land first.
- **UNKNOWN: how much of the misunderstanding layer is attributable to governance-context volume** (the §5.1 crowding hypothesis). Observable signature is present; a controlled measurement (same tasks, reduced-context mirror) has not been run.
- **UNCONFIRMED: that the 2026-07-18 release-seam remediation has been exercised end-to-end on a real tag.** The seam is wired and its components exist; no tag has been minted through it yet in this record.

Appendix A — Measurement commands (parameterised; every absence needled)

Placeholders: `<DB>` = the project's tracked SQLite workable-items SSoT · `<REG>` = the standing regression-guard registry (TSV) · `<SUITE>` = the regression-guard suite script · `<TAG>` = the release-tag script · `<PREBUILD>` = the pre-build verification suite · `<GATES>` = the constitution's gate-script directory · `<CONST>` = the canonical constitution rule file · `<MIRROR>` = the canonical agent-facing mirror file. All run 2026-07-22 in the consuming project's Track-1 checkout; historical figures cite that project's dated forensic corpus (2026-07-17 fix-lifecycle formula + status-custody proposal; 2026-07-19 QA audit; 2026-07-20→22 measurement-trap catalogue).

Ref	Command (parameterised)	Needle
M1	`sqlite3 -readonly <DB> "SELECT COUNT(DISTINCT atm_id) FROM items"` → 619; terminal: `...WHERE status IN (<3 terminal values>)` → 108	non-zero corpus proves path sighted
M2	terminal-status ids `NOT IN (SELECT atm_id FROM item_history)` → 52	history table non-empty (682 rows by type-sum)
M3	`SELECT event_type, COUNT(*) FROM item_history GROUP BY event_type` → Reopened 25; Fixed 24 + Implemented 2 + Completed 25 = 51	same query returns Opened 287
M4	status='Reopened' → 26; minus ids with a Reopened event → 13	a known item with events joins correctly
M5	per-id `SUM(event_type='Reopened'), SUM(terminal)` → top row 4/0	second row returns 2/0 (distinct id)
M6	`SELECT by, COUNT(*) FROM item_history WHERE event_type='Reopened' GROUP BY by` → AI 25	reason column returns 3 distinct values
M7	done-claiming: 4-status query → 187; join lead-key + alias columns of `<REG>` → 14	known-registered id present in join set
M8	`awk -F'\t' '!/^#/&&NF>1{print $1}' <REG> | sort -u | wc -l` → 59; item-shaped: `grep -cE '^(A-Z]+-[0-9]+'` -class → 37	`cat -A` proves 6 `^I` / row schema
M9	read `<SUITE>` tail: FAIL→exit 1; PENDING→exit 3 (landed per `git log -- <SUITE>` 2026-07-18); prior live run figures from the dated forensic doc + its preserved evidence file	unknown-verdict branch fails-safe to FAIL (read)

Ref	Command (parameterised)	Needle
M10	`grep -n regression_guard <TAG>` + read of the verdict-coverage block; `git log -- <verdict-coverage script>` → 2026-07-18	needle: known token in `<TAG>` returns 5 hits
M11	`ls <custody sweep path>` → absent; `grep -rln CM-STATUS-CUSTODY scripts/ <GATES>/..` → 0; `ls <ratchet file>` → absent	`ls <TAG>` resolves; grep needle on same tree returns the mutation-tool file
M12	`wc -l <CONST>` → 10,735; `grep -oE '§11\.4\.[0-9]+' <CONST> | sort -u | wc -l` → 220 (max 224); `grep -c "gate-code = separate work item"` → 85; named gates `grep -oE 'CM-[A-Z0-9-]+' | sort -u` → 413; minus `<PREBUILD>` set → 254; minus `<GATES>` set (20 gates) → 241	known-implemented gate in `<PREBUILD>` set = 1; known-deferred gate = 0 everywhere scanned
M13	`grep -c '^\*\*§11\.4\.<a> —' <MIRROR>` → 2 for two anchors	two sibling anchors return exactly 1
M14	consuming project's 2026-07-19 QA-audit doc, §0–§A (verdict table: 1 CLOSED-VALIDATED / 25)	the audit's own DB queries are reproduced in-doc
M15	consuming project's 2026-07-17 recurrence-map doc §0 (15→~9; 4 recurrences; 1 verified-distinct)	corroborated by the 07-19 deep-dive's git-provenance spine
M16	consuming project's 2026-07-17 formula doc §1.4 (evidence files preserved beside it)	commit totals reproduced from `git log` in that doc

Ref	Command (parameterised)	Needle
M17	consuming project's measurement-trap catalogue (Revision 1 + §5–§7), 22 numbered traps	each trap entry carries its own needle tran-script
M18	consuming project's unwired-oracle + evidence-init defect memos (behavioural census 68/24/12; 79 guards)	census method + scope bounds stated in-memo
M19	consuming project's 2026-07-22 README-reachability audit (1,953→422 orphans; §6 item 1: the two entry-point mandates "contradict each other today")	walker script + full reachable/ orphan lists on disk beside it

Instrument notes for reproducers: do not use `find -newermt` on this host (proven blind — count-before/sleep/count-after instead); do not write `\|` in any ERE (host grep dialect treats it as a literal); never read a pipeline's exit status after a formatter; never trust a background wrapper's exit code over the produced artifact.

Appendix B — Anti-bluff certification

Every number in §2–§5 was either measured this session with the Appendix-A command (and its needle) or is cited to a dated forensic document in the consuming project's corpus that itself carries preserved evidence files; the two sources are distinguished throughout. No PASS is claimed for anything. This document proposes no rule text and edits no governance file; its §5 recommendations are analysis, not landed mechanism — by the corpus's own §11.4.205 standard they carry no force until someone implements their seams.